MODULE

# 16

# Testing with pytest

Prove your code works, change it without fear, and pass the "how do you test?" interview question.

YOU WILL LEARN

| | |
|---|---|
| pytest basics | `test_01_basics.py` |
| Fixtures & parametrize | `test_02_fixtures_and_parametrize.py` |
| Mocks, fakes, monkeypatch | `test_03_mocking.py` |
| TDD: test first | `test_04_tdd.py` |
| Property tests & coverage | `test_05_hypothesis_and_coverage.py` |
| Interview Q&A | `INTERVIEW.md` |

WHY TESTING

# Why write tests at all?

## THE PROBLEM

You fix a discount bug on Friday. On Monday, refunds are broken: your fix changed a function that refunds also used.

Clicking through the app by hand after every change doesn't scale.

## THE IDEA

**Automated tests check the behaviour in seconds, every time. They are the safety net that lets you change code with confidence.**

PICTURE IT TESTING

# The testing pyramid

▲ Many **small, fast** tests at the bottom, fewer **slow, broad** ones at the top. Each layer catches different bugs.

**unit tests (~70%)**
one function, no DB, no network: milliseconds

→

**integration (~20%)**
your code + real DB / API client

→

**end-to-end (~10%)**
the whole app through HTTP or a browser

Interviewers love this picture. Name the layers and say what you test in each.

TRY IT PYTEST BASICS

# A test is a function that asserts

```python
import pytest

def price_with_vat(net, rate=0.19):
    if net < 0:
        raise ValueError("net price can't be negative")
    return round(net * (1 + rate), 2)

def test_adds_vat():
    assert price_with_vat(100) == 119.0

def test_custom_rate():
    assert price_with_vat(100, rate=0.07) == 107.0

def test_negative_price_is_rejected():
    with pytest.raises(ValueError, match="negative"):
        price_with_vat(-1)
```

OUTPUT

```
test_adds_vat PASSED
test_custom_rate PASSED
test_negative_price_is_rejected PASSED
3 passed
```

1. Files **test_*.py**, functions **test_***: pytest finds them.
2. A plain **assert**; pytest shows both values on failure.
3. **pytest.raises** tests the error path too.

TRY IT FIXTURES AND PARAMETRIZE

# Reuse setup, test many cases

```python
import pytest

class Cart:
    def __init__(self): self.items = []
    def add(self, price, qty=1): self.items.append(price * qty)
    def total(self): return sum(self.items)

@pytest.fixture
def cart():                          # fresh setup for EVERY test
    c = Cart()
    c.add(10, qty=2)
    return c

def test_total(cart):
    assert cart.total() == 20

@pytest.mark.parametrize("price, qty, expected", [
    (5, 1, 25), (5, 2, 30), (0, 9, 20),
])
def test_add(cart, price, qty, expected):
    cart.add(price, qty)
    assert cart.total() == expected
```

OUTPUT

```
test_total PASSED
test_add[5-1-25] PASSED
test_add[5-2-30] PASSED
test_add[0-9-20] PASSED
4 passed
```

1. A **fixture** is requested by its **name** as an argument.
2. **parametrize**: one test, many inputs.
3. Shared fixtures live in **conftest.py**.

COMMON MISTAKE MOCKING

# Tests that call the real world

✗ WRONG

```python
def test_weather():
    # calls the REAL API: slow, flaky,
    # needs internet, may cost money
    assert get_temperature("Augsburg") > -50
```

The test fails when the Wi-Fi drops or the API changes, and you can't test "API is down" at all.

✓ RIGHT

```python
def test_weather(monkeypatch):
    def fake_get(url, **kw):
        return FakeResponse({"temp": 21.5})
    monkeypatch.setattr("weather.requests.get", fake_get)
    assert get_temperature("Augsburg") == 21.5
```

Replace the **boundary** (HTTP, DB, clock) with a fake. Fast, repeatable, and you can simulate errors.

TRY IT MOCKING

# unittest.mock: check how it was called

```python
from unittest.mock import Mock

def notify_if_low(stock, item, mailer):
    if stock < 5:
        mailer.send("ops@shop.de", f"{item} low: {stock} left")

mailer = Mock()
notify_if_low(3, "lamp", mailer)
mailer.send.assert_called_once_with("ops@shop.de", "lamp low: 3 left")
print("called:", mailer.send.call_args)

mailer = Mock()
notify_if_low(50, "lamp", mailer)
mailer.send.assert_not_called()
print("not called when stock is fine")

flaky = Mock(side_effect=ConnectionError("timeout"))
try:
    flaky()
except ConnectionError as e:
    print("simulated failure:", e)
```

OUTPUT

```
called: call('ops@shop.de', 'lamp low: 3 left')
not called when stock is fine
simulated failure: timeout
```

1. A **Mock** accepts any call and records it.
2. **side_effect** simulates errors.
3. Mock **where it's used**: patch **"weather.requests.get"**, not "requests.get".

STEP BY STEP TDD

# Test-driven development: red, green, refactor

| STEP | YOU DO | RESULT |
|---|---|---|
| 1 RED | write ONE small failing test for the next behaviour | the test fails (good!) |
| 2 GREEN | write the simplest code that passes | all tests pass |
| 3 REFACTOR | clean up names and duplication | tests stay green |
| repeat | the next behaviour | the design grows in small, safe steps |

Lesson **test_04_tdd.py** builds **slugify()** exactly like this, one test at a time.

TRY IT PROPERTY-BASED TESTING

# Let hypothesis invent the inputs

```python
from hypothesis import given, strategies as st

def slugify(text):
    return "-".join(text.lower().split())

@given(st.text())
def test_slug_has_no_spaces(text):
    assert " " not in slugify(text)

@given(st.text())
def test_slug_is_stable(text):          # slugify twice = once
    assert slugify(slugify(text)) == slugify(text)
```

OUTPUT

```
test_slug_has_no_spaces PASSED
test_slug_is_stable PASSED
2 passed
```

1. You state a **rule**; hypothesis tries hundreds of inputs.
2. It finds edge cases you'd never type: tabs, emoji, empty strings.
3. Coverage: **pytest --cov=src** shows untested lines.

IN REAL JOBS TESTING

# Testing at work

### API tests with TestClient

```
def test_create_order(client):
    r = client.post("/orders", json={"item": "lamp"})
    assert r.status_code == 201
```

### A test database per run

```
@pytest.fixture
def db(): engine = create_engine("sqlite://"); ...
```

### Django tests

```
class OrderTests(TestCase):
    def test_list(self): ...
```

### CI runs everything

```
- run: pytest --cov=src --cov-fail-under=85
```

INTERVIEW SAY THIS IN THE INTERVIEW

# “How do you test code that calls an external API or database?”

- Keep the external call behind a small boundary (a client class or repository).
- In **unit** tests, replace it with a **fake** or a **Mock** (monkeypatch / mock.patch where it’s **used**).
- Simulate failures too: timeouts, 500s, empty results.
- In **integration** tests, use a real test database (SQLite in memory or a Postgres container).
- Run a few **end-to-end** tests against the deployed staging app.

> **Follow-up they ask next:** ”Mock vs stub vs fake?” (a stub returns canned answers; a fake is a working light version, like an in-memory repo; a mock also records and verifies calls).

INTERVIEW SAY THIS IN THE INTERVIEW

# “What is a pytest fixture? What is conftest.py?”

- A fixture is a function that provides **setup** (and optional teardown) for tests that ask for it by name.
- Code after **yield** runs as teardown: close connections, delete temp files.
- **scope** controls reuse: function (default), module, session.
- **conftest.py** holds fixtures shared by every test in its folder; pytest loads it automatically.

```python
@pytest.fixture(scope="session")
def engine():
    eng = create_engine("sqlite://")
    Base.metadata.create_all(eng)
    yield eng
    eng.dispose()
```

**Follow-up they ask next:** ”What’s a good coverage percentage?” (80-90% is common, but 100% coverage doesn’t mean correct code: test behaviour and edge cases, not lines).

RECAP REMEMBER THESE

# Module 16 in 6 lines

1. Pyramid: many **unit**, some **integration**, few **end-to-end** tests.
2. **assert** + **pytest.raises** for the happy and the error path.
3. **Fixtures** for setup, **parametrize** for many cases, **conftest.py** to share.
4. Fake the **boundaries** (HTTP, DB, time); patch where it's used.
5. TDD: **red, green, refactor** in tiny steps.
6. **hypothesis** finds edge cases; coverage finds untested code.

PRACTICE NOW

**Do exercises/ex16_test_password.py: write the tests for a password checker.**